7.2 Catching changes the path through the program

Keep Food.php and Dessert.php from 7.1. A catch gives the caller an alternative path when a particular operation cannot complete. Our first action will be to report the rejected operation and continue only with work that remains valid.

Catch the failed construction and update

Replace `public/tutorials/class7/index.php`:

```php
<?php
declare(strict_types=1);

use App\Tutorials\Class7\Dessert;
use App\Tutorials\Class7\Food;

require_once __DIR__ . '/../../../vendor/autoload.php';

echo new Food('Apple'), '<br>', PHP_EOL;

try {
    echo 'Before construction', '<br>', PHP_EOL;
    $dessert = new Dessert('Berry crumble', -1);
    echo $dessert, '<br>', PHP_EOL;
    echo 'Construction succeeded', '<br>', PHP_EOL;
} catch (Exception $e) {
    echo 'Construction rejected: ', $e->getMessage(), '<br>', PHP_EOL;
}
echo 'After the construction attempt', '<br>', PHP_EOL;

$existing = new Dessert('Oat biscuit', 250);
try {
    $existing->setCalories(-1);
    echo 'Update succeeded', '<br>', PHP_EOL;
} catch (Exception $e) {
    echo 'Update rejected: ', $e->getMessage(), '<br>', PHP_EOL;
}
echo 'Existing calories: ', $existing->getCalories(), '<br>', PHP_EOL;
```

Run the same CLI command from 7.1. Expect:

```
Food: Apple
Before construction
Construction rejected: Calories cannot be negative.
After the construction attempt
Update rejected: Calories cannot be negative.
Existing calories: 250
```

The catch variable `$e` refers to the thrown object. getMessage returns its message. These deliberately controlled messages are useful for our local walkthrough. Later we will choose approved public messages instead of displaying arbitrary exception details.

The statements between the throw and the end of try do not resume. Construction succeeded and Update succeeded never appear. Execution continues after the matching catch unless that block returns or throws again. Do not put an unconditional success message after catch.

What did the catch repair?

Nothing in the model was repaired. Failed construction still produced no new dessert. We do not read `$dessert` afterward. The existing dessert still has 250 calories because its setter rejected -1 before assignment. Catching alone does not guarantee that an object is unchanged: the order of the setter's statements gives us that guarantee for this property.

Exceptions also do not roll back a sequence of earlier successful changes. If you successfully change a name and then fail to change calories, the name change remains unless you explicitly restore it.

Our current response is useful for a teaching trace, but a real page needs a decision: tell someone how to correct input, substitute an acceptable optional value, or stop the failed task. Preventing a fatal error is only the first step.

Verify both paths

1. Change the failed constructor's -1 to 500: its description and success line now appear. The update still fails independently.
2. Change the setter's -1 to 300: the update success line appears and the final value becomes 300.
3. Restore both -1 arguments before continuing.